

Lab 7: Database Connectivity

ส่วนที่ 1: Software Design & Principles Explanation (เขียนอธิบาย)

GRASP Patterns และการแบ่งหน้าที่ของคลาส

- **Entity** : Game.java ทำหน้าที่เป็น **Information Expert** เพราะเก็บข้อมูลเกม (title, genre, price, discountType ฯลฯ)
- **Repository** : GameRepository.java ใช้หลัก **Indirection** โดยเป็นตัวกลางระหว่าง Service กับ Database (PostgreSQL)
- **Service** : GameService.java ใช้หลัก **High Cohesion** เพราะรวม business logic เช่นการจัดการเกม และ **Low Coupling** เพราะไม่ขึ้นตรงกับ Controller หรือ Repository
- **Controller** : GameController.java ใช้ **Controller Pattern** ทำหน้าที่รับ HTTP Request และส่งต่อไปยัง Service

SOLID Principles

- **SRP (Single Responsibility Principle)** : แต่ละคลาสมีหน้าที่เฉพาะ เช่น Entity เก็บข้อมูล, Repository จัดการ persistence, Service จัดการ business logic, Controller จัดการ request/response
- **OCP (Open/Closed Principle)** : Strategy Pattern ทำให้เพิ่มส่วนลดใหม่ได้โดยไม่ต้องแก้โค้ดเดิม
- **LSP (Liskov Substitution Principle)** : ทุก Strategy (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) สามารถแทนที่กันได้ผ่าน interface DiscountStrategy
- **ISP (Interface Segregation Principle)** : DiscountStrategy เป็น interface ที่แยกเฉพาะการคำนวณส่วนลด ไม่บังคับ method ที่ไม่จำเป็น
- **DIP (Dependency Inversion Principle)** : Service และ Controller พึ่งพา abstraction (Repository, Strategy) มากกว่า implementation

Strategy Pattern

- ใช้ DiscountStrategy เป็น interface
- มี implementation หลายแบบ:
 - NoDiscountStrategy → คำนวณราคาเต็ม
 - StudentDiscountStrategy → ลด 10%

- SeasonalSaleStrategy → ลด 20%
- DiscountContext : เลือก strategy ตาม discountType ของเกม
- **ประโยชน์:** รองรับ **OCP** : สามารถเพิ่มส่วนลดใหม่ได้โดยไม่ต้องแก้ไขโค้ดเดิม

Layered Architecture

- **Controller Layer** : รับ request จาก browser
- **Service Layer** : จัดการ business logic เช่นการคำนวณส่วนลด
- **Repository Layer** : ติดต่อกับ Database
- **Entity Layer** : เก็บข้อมูลเกม
- **ประโยชน์:**
 - **Low Coupling** : แต่ละ layer แยกจากกัน ไม่พึ่งพาโดยตรง
 - **High Cohesion** : แต่ละ layer มีหน้าที่ชัดเจน

Execution Flow

1. Browser ส่ง HTTP Request เช่น GET /games
2. **Controller** (GameController) รับ request เรียก GameService
3. **Service** (GameService) ดึงข้อมูลจาก GameRepository
4. **Repository** (GameRepository) ติดต่อ PostgreSQL → คืนข้อมูลเกม
5. **Service** ใช้ DiscountContext เลือก Strategy ที่เหมาะสม คำนวณราคาหลังส่วนลด
6. **Controller** ส่งข้อมูลไปยัง Thymeleaf Template (list.html)
7. Browser แสดงผลรายการเกมพร้อมราคาหลังส่วนลด

ส่วนที่ 2: Code Implementation & Explanation

1.Entity Layer – Game . java

@Entity

```
public class Game {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String title;
```

```
    private String genre;
```

```
    private double price;
```

```
    private String discountType; // ใช้เลือก Strategy
```

```
    // Constructor, Getter, Setter
```

```
}
```

Entity ทำหน้าที่เก็บข้อมูลเกม

ใช้หลัก **Information Expert** เก็บข้อมูลที่เกี่ยวข้องกับตัวเอง

2.Repository Layer – GameRepository.java

@Repository

```
public interface GameRepository extends JpaRepository<Game, Long> {
```

```
}
```

Repository เป็นตัวกลางเชื่อมกับ Database (PostgreSQL)

ใช้หลัก **Indirection** Controller/Service ไม่ต้องติดต่อ DB โดยตรง

3.Strategy Package

```
public interface DiscountStrategy {  
    double applyDiscount(double price);  
}
```

```
public class NoDiscountStrategy implements DiscountStrategy {  
    @Override  
    public double applyDiscount(double price) {  
        return price;  
    }  
}
```

```
public class StudentDiscountStrategy implements DiscountStrategy {  
    @Override  
    public double applyDiscount(double price) {  
        return price * 0.9; // ลด 10%  
    }  
}
```

```
public class SeasonalSaleStrategy implements DiscountStrategy {  
    @Override  
    public double applyDiscount(double price) {  
        return price * 0.8; // ลด 20%  
    }  
}
```

```
public class DiscountContext {  
    private final DiscountStrategy strategy;  
  
    public DiscountContext(DiscountStrategy strategy) {
```

```

        this.strategy = strategy;
    }

    public double getDiscountedPrice(double price) {
        return strategy.applyDiscount(price);
    }
}

```

ใช้ Strategy Pattern สามารถเพิ่มส่วนลดใหม่ได้โดยไม่แก้โค้ดเดิม (OCP)

4.Service Layer – GameService.java

```

@Service
public class GameService {
    private final GameRepository gameRepository;

    @Autowired
    public GameService(GameRepository gameRepository) {
        this.gameRepository = gameRepository;
    }

    public List<Game> getAllGames() {
        return gameRepository.findAll();
    }

    public Game saveGame(Game game) {
        return gameRepository.save(game);
    }

    public double calculateDiscount(Game game) {
        DiscountStrategy strategy;
        switch (game.getDiscountType()) {

```

```

        case "student":
            strategy = new StudentDiscountStrategy();
            break;
        case "seasonal":
            strategy = new SeasonalSaleStrategy();
            break;
        default:
            strategy = new NoDiscountStrategy();
    }
    DiscountContext context = new DiscountContext(strategy);
    return context.getDiscountedPrice(game.getPrice());
}
}

```

ใช้ **Constructor Injection** ทำให้ field เป็น final และ test ได้ง่าย
Service รวม business logic เช่นการคำนวณส่วนลด

5.Controller Layer – GameController.java

```

@Controller
@RequestMapping("/games")
public class GameController {
    private final GameService gameService;

    @Autowired
    public GameController(GameService gameService) {
        this.gameService = gameService;
    }

    @GetMapping
    public String listGames(Model model) {
        List<Game> games = gameService.getAllGames();
    }
}

```

```

        model.addAttribute("games", games);
        return "games/list";
    }

```

```

@PostMapping("/add")
public String addGame(@ModelAttribute Game game) {
    gameService.saveGame(game);
    return "redirect:/games";
}

```

```

@GetMapping("/{id}/discount")
@ResponseBody
public String getDiscount(@PathVariable Long id) {
    Game game = gameService.getAllGames().stream()
        .filter(g -> g.getId().equals(id))
        .findFirst()
        .orElseThrow();
    double discountedPrice = gameService.calculateDiscount(game);
    return "Discounted Price: " + discountedPrice;
}
}

```

Controller ใช้ **Constructor Injection** เพื่อรับ Service

ทำหน้าที่รับ HTTP Request ส่งต่อไปยัง Service และ คืนผลลัพธ์

ส่วนที่ 3: Web Application & Database Screenshots

เพิ่มเกมใหม่ ส่วนลดนักศึกษา 10%

 CP353002 — Lab 7

[← กลับไปรายการเกม](#)

+

เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม

Elden Ring (673380605-0 SEC 3)

แนวเกม (Genre)

Action RPG

แพลตฟอร์ม (Platform)

PC/PS5

☆ คะแนน (Rating)

9.8

🏷 ราคาปกติ (บาท)

1790

% ส่วนลด (Strategy)

ส่วนลดนักศึกษา (10%)

📅 วันวางจำหน่าย (Release Date)

07/08/2026

บันทึก

ยกเลิก

เพิ่มเกม ส่วนลดนักศึกษา 20%

 CP353002 — Lab 7

[← กลับไปรายการเกม](#)

+

เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม

Roblox

แนวเกม (Genre)

Action

แพลตฟอร์ม (Platform)

PC

☆ คะแนน (Rating)

9

🏷 ราคาปกติ (บาท)

1299

% ส่วนลด (Strategy)

ส่วนลดเทศกาล (20%)

📅 วันวางจำหน่าย (Release Date)

07/08/2026

บันทึก

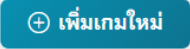
ยกเลิก

หน้าจอสื่อรายการเกมทั้งหมด

CP353002 — Lab 7: Spring Boot + JPA + JDBC + PostgreSQL

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete



เพิ่มเกม "Roblox" สำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Elden Ring (673380605-0 SEC 3)	Action RPG	PC/PS5	★ 9.8	07/08/2026	ส่วนลดนักศึกษา (10%)	1611.00 1790.00	 
2	Roblox	Action	PC	★ 9.0	07/08/2026	ส่วนลดเทศกาล (20%)	1039.20 1299.00	 

 ทั้งหมด 2 เกม

หน้าจอแก้ไขเกม (Update) แสดงฟอร์มแก้ไขข้อมูลเกม แก้ไขวันที่เป็น 8/8/2026

CP353002 — Lab 7

 กลับไปรายการเกม

 แก้ไขข้อมูลเกม

แก้ไขข้อมูลเกม: Elden Ring (673380605-0 SEC 3)

 ชื่อเกม

Elden Ring (673380605-0 SEC 3)

 แนวเกม (Genre)

Action RPG

 แพลตฟอร์ม (Platform)

PC/PS5

 คะแนน (Rating)

9.8

 ราคาปกติ (บาท)

1790.0

 ส่วนลด (Strategy)

ส่วนลดนักศึกษา (10%)

 วันวางจำหน่าย (Release Date)

08/08/2026

 บันทึก

 ยกเลิก

รายการเกมทั้งหมด

จัดการข้อมูลเกมในระบบ — Create, Read, Update, Delete

+ เพิ่มเกมใหม่

✓ อัปเดตข้อมูลเกม "Elden Ring (673380605-0 SEC 3)" สำเร็จ

#	ชื่อเกม	แนวเกม	แพลตฟอร์ม	คะแนน	วันวางจำหน่าย	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	จัดการ
1	Roblox	Action	PC	★ 9.0	07/08/2026	ส่วนลดเทศกาล (20%)	1039.20 1299.00	 
2	Elden Ring (673380605-0 SEC 3)	Action RPG	PC/PSS	★ 9.8	08/08/2026	ส่วนลดนักศึกษา (10%)	1611.00 1790.00	 

ทั้งหมด 2 เกม

หน้าจอยืนยันลบ + ผลลัพธ์หลังลบ (Delete)

← กลับไปรายการเกม

 ยืนยันการลบ

คุณแน่ใจหรือไม่?

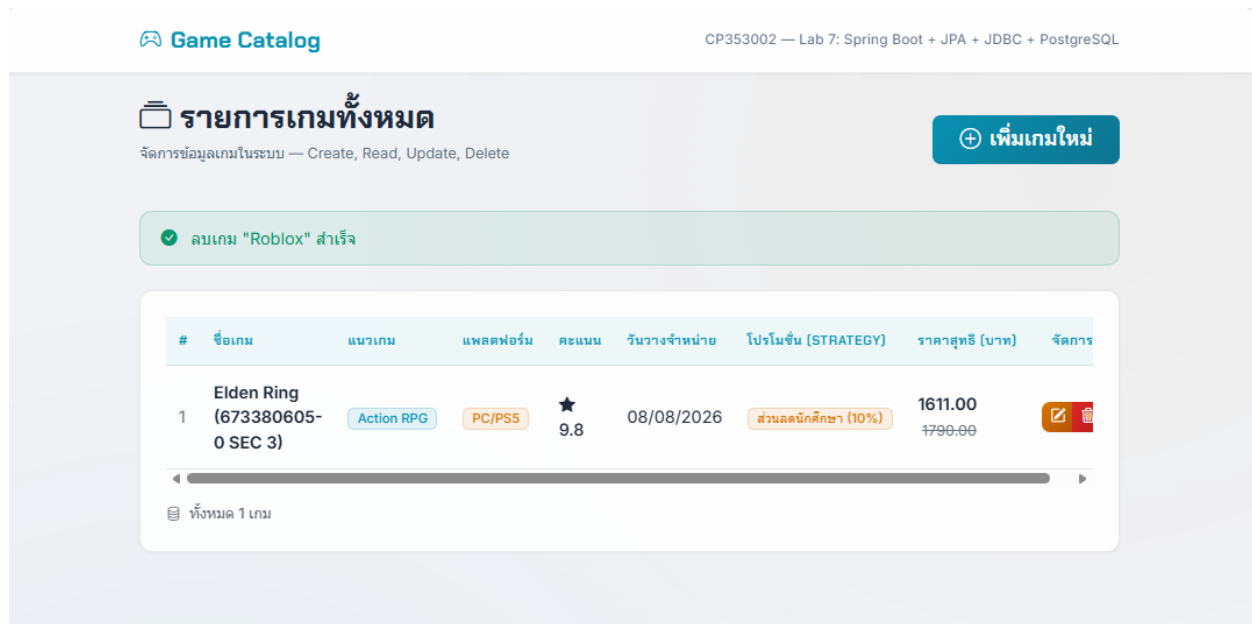
การดำเนินการนี้จะลบข้อมูลเกมออกจากระบบอย่างถาวร

ชื่อเกม	Roblox
แนวเกม	Action
แพลตฟอร์ม	PC
คะแนน	★ 9.0
ราคา	1299.00 บาท

 ยืนยันลบ

← ยกเลิก

เกม Roblox หายไป



หน้าจอ Database (pgAdmin) แสดงข้อมูลจริงในตาราง games ที่มีรหัสนักศึกษานั้นที่กอยู่

Data Output Messages Notifications

Showing rows: 1 to 2 of 1 Page No: 1

	id [PK] bigint	discount_type character varying (255)	genre character varying (255)	platform character varying (255)	price double precision	rating double precision	release_date date	title character varying (255)
1	1	STUDENT	Action RPG	PC/PS5	1790	9.8	2026-08-07	Elden Ring (673380605-0 SEC...
2	2	SEASONAL	Action	PC	1299	9	2026-08-07	Roblox